Generative Adversarial Imitation Learning

Εθνικό Μετσόβιο Πολυτεχνείο
Σχολή Ηλεκτρολόγων Μηχανικών και Μηχανικών Υπολογιστών
Νευροασαφής Έλεγχος και Εφαρμογές
Ονοματεπώνυμο: Ιωάννης Λίτσος
ΑΜ: 03119135
11ο εξάμηνο

Φεβρουάριος 2025

Abstract

This project explores the application of Generative Adversarial Imitation Learning (GAIL) across various control tasks. We evaluate GAIL's performance in imitating expert policies generated by Trust Region Policy Optimization (TRPO), Proximal Policy Optimization (PPO), and Generalized Advantage Estimation (GAE) across four distinct OpenAI Gymnasium environments: CartPole, Pendulum, BipedalWalker, and LunarLander. Our experimental results demonstrate that GAIL successfully captures expert behaviors with varying degrees of fidelity depending on task complexity and expert policy stability. The study reveals that while GAIL achieves rapid convergence in simpler environments, more complex tasks require substantially more iterations for effective learning. Additionally, we observe that the quality and stability characteristics of expert demonstrations significantly influence GAIL's imitation performance. The research provides valuable insights into the practical implementation of GAIL and its effectiveness in transferring complex behaviors from experts to learning agents, while also highlighting important considerations regarding computational requirements and training stability.

Chapter 1

Introduction

1.1 Reinforcement Learning

Reinforcement Learning (RL) is a subfield of machine learning that focuses on how agents take actions in an environment to maximize cumulative rewards. The RL framework is typically formulated as a Markov Decision Process (MDP), defined by the tuple (S, A, P, R, γ), where:

- S is the set of states, representing different situations an agent can encounter.
- A is the set of actions, defining possible decisions the agent can make.
- $P(s'|s, a)$ is the transition probability function, which describes the probability of reaching state s' given the current state s and action a.
- $R(s, a)$ is the reward function, providing feedback on the desirability of state-action pairs.
- $\gamma \in [0, 1]$ is the discount factor, determining the importance of future rewards, where a value close to 1 favors long-term rewards over immediate rewards.

The agent interacts with the environment in discrete time steps, selecting actions according to a policy $\pi(a|s)$, receiving rewards, and transitioning to new states. The objective is to find an optimal policy π^* that maximizes the expected return:

$$J(\pi) = \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t)\right]. \tag{1.1}$$

Central to RL are value functions, which estimate the long-term reward an agent can expect to receive. The state-value function $V^\pi(s)$ represents the

expected return when starting in state s and following policy π thereafter:

$$V^{\pi}(s) = \mathbb{E}_{\pi}\left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) | s_0 = s\right]. \tag{1.2}$$

Similarly, the action-value function $Q^{\pi}(s, a)$ represents the expected return when taking action a in state s and following policy π thereafter:

$$Q^{\pi}(s, a) = \mathbb{E}_{\pi}\left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) | s_0 = s, a_0 = a\right]. \tag{1.3}$$

Several approaches exist for solving RL problems, broadly categorized into:

- **Value-based methods:** These methods focus on estimating the value of actions or states through value functions, such as Q-learning and Deep Q-Networks (DQN) [5], which iteratively update the Q-value function using the Bellman equation.
- **Policy-based methods:** These directly optimize the policy function without necessarily requiring a value function, often using techniques such as as Policy Gradient, Trust Region Policy Optimization (TRPO) [9], or Proximal Policy Optimization (PPO) [8].
- **Inverse Reinforcement Learning (IRL) [6, 1]:** These methods aim to recover the underlying reward function of expert demonstrations, providing a bridge between traditional RL and imitation learning approaches.

Reinforcement learning has demonstrated remarkable success across various domains, including robotics, game playing (such as AlphaGo and OpenAI Gymnasium environments [2]), autonomous driving, and financial trading. However, several fundamental challenges persist in the field. The sample inefficiency of many RL algorithms often requires millions of interactions with the environment to learn effective policies. The exploration-exploitation dilemma remains a significant challenge, particularly in sparse reward settings. Perhaps most critically, the difficulty of reward specification, which often requires manual engineering and may not capture the true objectives of complex tasks, has emerged as a fundamental limitation. This challenge of reward design, combined with the sample efficiency problem, motivates the development of alternative approaches like imitation learning and, specifically, Generative Adversarial Imitation Learning (GAIL) [4], which we will explore in subsequent sections.

1.2 Traditional RL Algorithms

Traditional reinforcement learning algorithms have evolved significantly over the past decades, with each approach offering distinct advantages and limitations. These algorithms can be broadly categorized into three main families:

value-based methods, policy-based methods, and inverse reinforcement learning approaches.

Value-based methods, exemplified by Q-learning and its deep learning variant DQN, estimate the optimal action-value function $Q^*(s, a)$ to derive a policy. The Q-function is updated iteratively using the Bellman equation:

$$Q(s, a) \leftarrow Q(s, a) + \alpha[R(s, a) + \gamma \max_{a'} Q(s', a') - Q(s, a)] \tag{1.4}$$

where α is the learning rate. While these methods have achieved remarkable success in discrete action spaces, such as Atari games, they face challenges in continuous action domains due to the difficulty of maximizing over continuous action spaces. Additionally, they often require substantial environmental interactions to converge to optimal policies.

Policy-based methods directly optimize the policy parameters θ to maximize the expected return. The policy gradient theorem provides the foundation for these approaches:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}[\nabla_\theta \log \pi_\theta(a|s) Q^{\pi_\theta}(s, a)] \tag{1.5}$$

Trust Region Policy Optimization (TRPO) improves upon vanilla policy gradient methods by ensuring stable policy updates through constrained optimization. TRPO formulates the policy improvement step as:

$$maximize_\theta \mathbb{E}_{s,a\sim\pi_{\theta_{\text{old}}}} \left[\frac{\pi_\theta(a|s)}{\pi_{\theta_{\text{old}}}(a|s)} A_{\theta_{\text{old}}}(s, a) \right] \tag{1.6}$$

$$\text{subject to } \mathbb{E}_{s\sim\rho_{\theta_{\text{old}}}}[D_{\text{KL}}(\pi_{\theta_{\text{old}}}(\cdot|s)||\pi_\theta(\cdot|s))] \leq \delta \tag{1.7}$$

where $A_{\theta_{\text{old}}}(s, a)$ is the advantage function estimating how much better action a is compared to the average action in state s, and D_{KL} represents the Kullback-Leibler divergence measuring the difference between old and new policies. The constraint ensures that the new policy stays within a "trust region" of radius δ around the old policy, preventing destructively large policy updates.

Proximal Policy Optimization (PPO) simplifies TRPO's approach while maintaining its benefits by replacing the constrained optimization with a clipped objective function:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t[\min(r_t(\theta)A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)A_t)] \tag{1.8}$$

where $r_t(\theta)$ denotes the probability ratio $\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$, and ϵ is a hyperparameter, typically set to 0.2. The clipping function ensures that the probability ratio stays within $[1 - \epsilon, 1 + \epsilon]$, effectively creating a trust region. This objective function has two key properties:

- For positive advantages ($A_t > 0$), the objective is clipped above at $1 + \epsilon$, discouraging the policy from moving too far in the direction of increasing the action probability.

- For negative advantages ($A_t < 0$), the objective is clipped below at $1 - \epsilon$, discouraging the policy from moving too far in the direction of decreasing the action probability.

Both algorithms typically employ a value function baseline to reduce variance in the policy gradient estimates:

$$V_\phi(s) \approx \mathbb{E}_{\pi_\theta}[\sum_{t=0}^{\infty} \gamma^t r_t | s_0 = s] \tag{1.9}$$

The advantage function is then estimated using the Generalized Advantage Estimation (GAE) [7]:

$$A_t = \delta_t + (\gamma\lambda)\delta_{t+1} + ... + (\gamma\lambda)^{T-t+1}\delta_{T-1} \tag{1.10}$$

where $\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$ is the temporal difference error, and λ is a hyperparameter controlling the bias-variance trade-off in the advantage estimation.

These algorithms have demonstrated remarkable success in continuous control tasks and robotics applications. PPO, in particular, has become one of the most widely used RL algorithms due to its simplicity, reliability, and strong performance across a wide range of tasks. However, both methods still face challenges:

- Sample efficiency remains a concern, often requiring millions of environment interactions for complex tasks.
- The performance can be sensitive to hyperparameter choices, particularly the learning rate and the trust region size (δ for TRPO, ϵ for PPO).
- The methods may struggle in environments with sparse rewards or when the reward function is poorly designed.

Inverse Reinforcement Learning (IRL) takes a fundamentally different approach by attempting to recover the reward function from expert demonstrations. Given a set of expert trajectories τ_E, IRL aims to find a reward function R that makes the expert behavior appear optimal:

$$R^* = argmax_R(\min_\pi -H(\pi) + \mathbb{E}_\pi[R(s,a)]) - \mathbb{E}_{\pi_E}[R(s,a)] \tag{1.11}$$

where $H(\pi)$ represents the entropy of the policy. While IRL addresses the challenge of reward specification, it introduces new computational complexities, as it requires solving the forward RL problem multiple times during the optimization process.

Each of these approaches faces distinct challenges. Value-based methods often struggle with sample efficiency and continuous action spaces. Policy-based methods, while more flexible, require extensive hyperparameter tuning and can be unstable during training. IRL methods, though theoretically appealing for

imitation learning, face computational challenges and may not scale well to complex tasks. These limitations, particularly in scenarios where reward specification is challenging or expert demonstrations are available, motivate the development of more efficient imitation learning approaches like GAIL, which we will explore after discussing the foundational concept of Generative Adversarial Networks.

1.3 Generative Adversarial Networks

Generative Adversarial Networks (GANs) [3] represent a groundbreaking approach to generative modeling that has revolutionized the field of machine learning. The core innovation of GANs lies in their adversarial training framework, where two neural networks compete against each other in a minimax game.

The GAN framework consists of two primary components: a generator network G and a discriminator network D. The generator aims to create synthetic data that resembles a target distribution, while the discriminator attempts to distinguish between real and generated samples. This adversarial relationship is formalized through the following objective function:

$$\min_G \max_D \mathbb{E}_{x \sim p_{\text{data}}}[\log D(x)] + \mathbb{E}_{z \sim p_z}[\log(1 - D(G(z)))] \tag{1.12}$$

where p_{data} represents the real data distribution and p_z is typically a simple noise distribution (e.g., Gaussian or uniform).

During training, the discriminator learns to maximize the probability of correctly classifying both real and generated samples, while the generator learns to minimize the probability of the discriminator correctly identifying generated samples. At convergence, the generator should produce samples that the discriminator cannot distinguish from real data, effectively learning to replicate the true data distribution.

The success of GANs in generating high-quality synthetic data has led to their adaptation for various tasks beyond traditional image generation. Of particular relevance to imitation learning is their ability to learn complex data distributions without explicit density estimation. This property proves valuable in the context of GAIL, where the goal is to learn a policy that generates trajectories matching those of an expert demonstrator.

However, GANs also present several training challenges:

- **Mode collapse:** The generator may learn to produce a limited variety of samples that consistently fool the discriminator, rather than capturing the full diversity of the target distribution.
- **Training instability:** The adversarial training process can be unstable, with the generator and discriminator potentially falling into destructive feedback loops.
- **Convergence difficulties:** Determining when the training has converged and finding the Nash equilibrium of the minimax game can be challenging.

These challenges have led to numerous architectural and training innovations, many of which influence the design of GAIL. The adversarial framework's ability to learn complex distributions without explicit modeling makes it particularly suitable for imitation learning, where the goal is to match the distribution of expert trajectories. This connection between GANs and imitation learning provides the foundation for GAIL's approach to policy learning, which we will explore in detail in subsequent sections.

Chapter 2

Methodology

This chapter presents a detailed examination of Generative Adversarial Imitation Learning (GAIL) [4], a groundbreaking approach to imitation learning that combines insights from inverse reinforcement learning (IRL) and generative adversarial networks (GANs). We begin by establishing the theoretical foundations that motivate GAIL's development, then proceed to describe its key components and implementation details.

2.1 Theoretical Foundations

2.1.1 Occupancy Measure

At its core, GAIL approaches imitation learning through the lens of matching occupancy measures between the expert and the learning agent. For a policy π, its occupancy measure $\rho_\pi : S \times A \to \mathbb{R}$ is defined as:

$$\rho_\pi(s, a) = \pi(a|s) \sum_{t=0}^{\infty} \gamma^t P(s_t = s|\pi) \tag{2.1}$$

This measure represents the distribution of state-action pairs that an agent encounters when navigating the environment using policy π. The occupancy measure allows us to rewrite the cost function as follows:

$$\mathbb{E}_\pi[c(s, a)] = \sum_{s,a} \rho_\pi(s, a)c(s, a) \tag{2.2}$$

A fundamental result in the field establishes that the set of valid occupancy measures $\mathcal{D} = \{\rho_\pi : \pi \in \Pi\}$ can be characterized by a set of affine constraints:

$$\mathcal{D} = \{\rho : \rho \geq 0 \text{ and } \sum_a \rho(s, a) = p_0(s) + \gamma \sum_{s',a} P(s|s', a)\rho(s', a) \quad \forall s \in S\} \tag{2.3}$$

where $p_0(s)$ is the initial state distribution and $P(s'|s,a)$ represents the environment dynamics.

2.1.2 Direct Policy Learning

Traditional IRL approaches attempt to recover a cost function that explains expert behavior, followed by a reinforcement learning step to extract a policy. GAIL's key insight is that this two-step process can be reformulated as a direct policy optimization problem. This reformulation stems from a crucial theoretical result: for any regularizer ψ, the policy obtained by running reinforcement learning on the cost function recovered by ψ-regularized IRL can be characterized as:

$$\mathrm{RL} \odot \mathrm{IRL}_{\psi}(\pi_E) = argmin_{\pi \in \Pi} - H(\pi) + \psi^*(\rho_\pi - \rho_{\pi_E}) \tag{2.4}$$

where ψ^* is the convex conjugate of ψ, and $H(\pi)$ is the γ-discounted causal entropy of policy π.

2.2 GAIL Framework

2.2.1 Adversarial Learning Formulation

GAIL introduces a novel cost regularizer ψ_{GA} that enables powerful imitation learning while maintaining tractability. This regularizer is defined as:

$$\psi_{GA}(c) = \begin{cases} \mathbb{E}_{\pi_E}[g(c(s,a))] & \text{if } c < 0 \\ +\infty & \text{otherwise} \end{cases} \tag{2.5}$$

where

$$g(x) = \begin{cases} -x - \log(1 - e^x) & \text{if } x < 0 \\ +\infty & \text{otherwise} \end{cases} \tag{2.6}$$

This regularizer leads to an optimization objective that can be expressed as:

$$\psi^*_{GA}(\rho_\pi - \rho_{\pi_E}) = \max_{D \in (0,1)^{S \times A}} \mathbb{E}_\pi[\log(D(s,a))] + \mathbb{E}_{\pi_E}[\log(1 - D(s,a))] \tag{2.7}$$

where D represents a discriminative classifier. This formulation draws a direct parallel to GANs, where the policy π acts as the generator, producing state-action pairs, and the discriminator D attempts to distinguish between the policy's and expert's state-action pairs. The objective measures the Jensen-Shannon divergence between occupancy measures.

2.2.2 Optimization Objective

The complete GAIL optimization problem can be expressed as:

$$\min_{\pi} \max_{D} \mathbb{E}_{\pi}[\log(D(s,a))] + \mathbb{E}_{\pi_E}[\log(1 - D(s,a))] - \lambda H(\pi) \tag{2.8}$$

where $\lambda \geq 0$ controls the strength of the entropy regularization term. This objective serves to minimize the Jensen-Shannon divergence between the policy's and expert's occupancy measures while maintaining sufficient policy entropy to prevent premature convergence to suboptimal solutions.

2.3 Implementation Details

In practice, both the policy π_θ and discriminator D_w are implemented as neural networks. The policy network takes state representations as input and outputs action distribution parameters, while the discriminator network takes state-action pair concatenations as input and outputs a single scalar in $(0, 1)$. Both networks typically employ an architecture with two hidden layers of 100 units each and tanh activations.

The GAIL training process alternates between two main steps. First, the discriminator update samples trajectories from the current policy and updates the discriminator parameters using the gradient:

$$\hat{\mathbb{E}}_{\tau_i}[\nabla_w \log(D_w(s,a))] + \hat{\mathbb{E}}_{\tau_E}[\nabla_w \log(1 - D_w(s,a))] \tag{2.9}$$

This optimization is performed using Adam with standard hyperparameters. Second, the policy update uses Trust Region Policy Optimization (TRPO) with the cost function $\log(D_w(s,a))$. This involves taking a KL-constrained natural gradient step using:

$$\hat{\mathbb{E}}_{\tau_i}[\nabla_\theta \log \pi_\theta(a|s) Q(s,a)] - \lambda \nabla_\theta H(\pi_\theta) \tag{2.10}$$

where

$$Q(\bar{s}, \bar{a}) = \hat{\mathbb{E}}_{\tau_i}[\log(D_w(s,a))|s_0 = \bar{s}, a_0 = \bar{a}] \tag{2.11}$$

The TRPO step is crucial for stable training, as it prevents large policy changes that could lead to training collapse. The KL constraint ensures that consecutive policies remain close in behavior space.

2.4 Theoretical Properties

Compared to traditional approaches, GAIL offers several advantages. Unlike behavioral cloning, it avoids compounding errors and provides better generalization to unseen states while being more robust to expert data sparsity. In contrast to classical IRL, it eliminates the need for explicit reward function recovery, offers better scalability to high-dimensional problems, and provides

a more efficient optimization procedure. When compared to imitation learning, GAIL removes the requirement for hand-crafted feature expectations, can capture arbitrary expert behaviors, and employs a more expressive policy class.

Chapter 3

Experimental Setup

3.1 Overview

This chapter presents the experimental framework used to evaluate the performance of Generative Adversarial Imitation Learning across various control tasks. Our experiments were designed to assess GAIL's ability to learn from expert demonstrations generated by different state-of-the-art reinforcement learning algorithms.

3.2 Environment Selection

We selected four distinct environments from OpenAI Gymnasium, each presenting unique challenges and complexity levels. These environments provide a diverse set of control problems ranging from simple balance tasks to complex locomotion challenges.

The Cart Pole environment (Figure 3.1) represents a classic control problem where a pole is attached to a cart moving on a frictionless track. The objective is to balance the pole by applying left or right forces to the cart. The system operates with a discrete action space consisting of two possible actions: pushing the cart left or right. The environment provides a 4-dimensional observation space that includes the cart position, cart velocity, pole angle, and pole angular velocity. An episode terminates if the pole angle exceeds ± 12 degrees or if the cart position exceeds ± 2.4 units, with a reward of $+1$ granted for each step the pole remains balanced.

The Pendulum (Figure 3.2) environment presents an inverted pendulum swingup problem where the goal is to swing a pendulum into an upright position and maintain it there. This environment features a continuous action space, where the action represents the torque applied to the pendulum in the range [-2.0, 2.0]. The observation space is 3-dimensional, consisting of the sine and cosine of the pendulum's angle and its angular velocity. The reward function considers the pendulum's angle from vertical, its angular velocity, and the

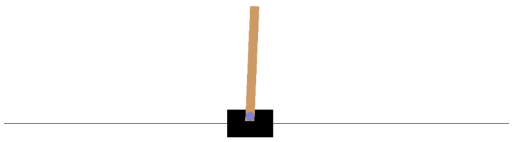


Figure 3.1: Cart Pole environment

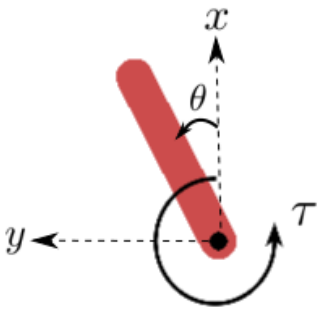


Figure 3.2: Pendulum environment

applied torque, with episodes lasting 200 time steps.

The Bipedal Walker environment (Figure 3.3) introduces a 4-joint walker robot that must learn to traverse terrain efficiently. This environment operates in a continuous action space, controlling motor speeds for four joints (hips and knees). The observation space is more complex, incorporating hull angle, joint positions, angular velocities, and lidar measurements. The reward structure encourages forward progress while penalizing falls and excessive motor usage. Episodes terminate if the hull makes contact with the ground.

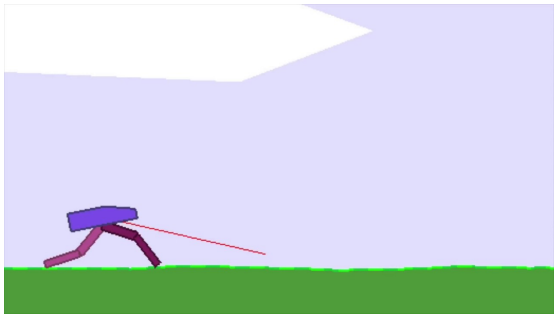


Figure 3.3: Bipedal Walker environment

The Lunar Lander environment (Figure 3.4) presents a trajectory optimization problem where an agent must land a spacecraft safely. It features a discrete action space with four possible actions: no operation, firing the left engine, main engine, or right engine. The observation space is 8-dimensional, encompassing position, velocity, angle, angular velocity, and leg contact flags. Rewards are based on distance to the landing pad, velocity, angle, and fuel usage, with episodes ending upon successful landing, crash, or leaving the viewport.

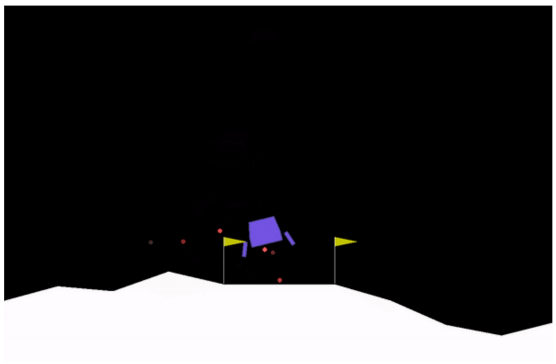


Figure 3.4: Lunar Lander environment

3.3 Expert Policy Generation

The generation of expert demonstrations was accomplished using three state-of-the-art reinforcement learning algorithms: Trust Region Policy Optimization (TRPO), Proximal Policy Optimization (PPO), and Generalized Advantage Estimation (GAE). Each algorithm brings distinct advantages to the policy learning process.

TRPO utilizes a trust region constraint to ensure stable policy updates, optimizing policies while limiting the KL divergence between consecutive policy iterations. This approach is particularly well-suited for continuous control tasks due to its conservative update strategy. PPO simplifies TRPO's approach using a clipped objective function, providing similar performance with better computational efficiency and showing particular effectiveness in environments with discrete action spaces. GAE combines value function estimation with policy optimization, providing a balanced trade-off between bias and variance in policy gradients, which helps reduce sample variance during training.

For each environment-algorithm combination, the expert policies were trained until convergence or until achieving specified performance thresholds: 500 for Cart Pole, -200 for Pendulum, 300 for Bipedal Walker, and 200 for Lunar Lander. After training, we collected trajectories from the expert policies to create demonstration datasets, with each trajectory consisting of state-action pairs that capture successful task completions.

3.4 Training Configuration

Our GAIL implementation maintained consistent hyperparameters across all experiments, with the exception of environment-specific iteration settings. Several key hyperparameters were kept constant across all environments: the lambda regularization parameter (λ), which controls the strength of the entropy regularization term, was set to 0.001; the maximum KL divergence was set to 0.01 to ensure stable policy updates; and the conjugate gradient damping parameter

was set to 0.1. Initial experiments with $\lambda = 0$ showed poorer performance across most environments, leading us to maintain the non-zero regularization for all subsequent experiments.

The number of iterations and steps per iteration varied according to the complexity of each task, as shown in Table 3.1.

Table 3.1: Environment-specific training parameters

Environment	Number of Iterations	Steps per Iteration
Cart Pole	200	2,000
Pendulum	500	2,000
Bipedal Walker	1,000	5,000
Lunar Lander	1,000	5,000

This variation in iteration counts and steps reflects the increasing complexity of the tasks, with simpler environments requiring fewer iterations compared to more complex ones.

3.5 Evaluation Methodology

To assess the performance of GAIL and compare it with expert policies, we employed mean reward per iteration during training as our primary metric. This metric was calculated by averaging the rewards obtained across all episodes within each iteration, providing insight into the learning progress and stability while allowing direct comparison with expert performance levels.

Throughout the training process, performance was tracked across the specified number of iterations for each environment, with regular logging intervals to generate learning curves. Convergence was assessed based on reward stability and comparison to expert performance, providing a comprehensive view of GAIL's ability to learn from and replicate expert behavior across different environments and demonstration sources.

Chapter 4

Results

4.1 Expert Policy Training

The training performance of TRPO, PPO, and GAE was evaluated across different control tasks, each presenting unique challenges. Figures 4.1-4.3 show the learning curves for these algorithms, with the solid lines representing mean rewards and the shaded areas indicating standard deviation.

4.1.1 CartPole-v1 Environment

The CartPole results (Figure 4.1) show rapid learning for all three algorithms, with convergence occurring within the first 25 iterations. All methods achieved the environment's maximum possible reward of 500, but with varying degrees of stability. GAE demonstrated the most consistent performance post-convergence, with notably smaller variance in its rewards. TRPO and PPO, while achieving similar peak performance, showed occasional significant drops in performance, particularly visible in the later iterations. This suggests that while all algorithms can master the relatively simple dynamics of the CartPole task, GAE's advantage estimation provides more robust policy updates for maintaining stable behavior.

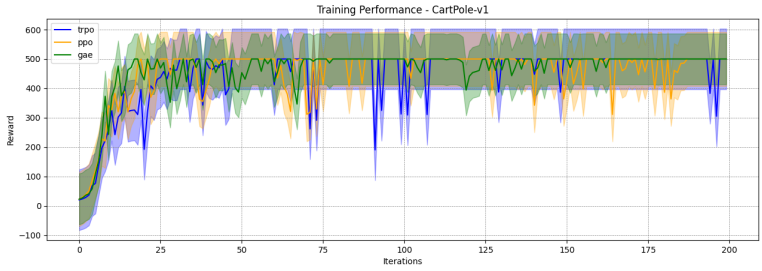


Figure 4.1: Training performance comparison on the CartPole-v1 environment

4.1.2 Pendulum-v0 Environment

The Pendulum environment (Figure 4.2) presents a continuous control problem, and the learning curves reveal interesting differences in algorithm behavior. TRPO showed the fastest initial learning rate, achieving better early-stage performance compared to PPO and GAE. However, by around iteration 200, all three algorithms converged to similar performance levels, with average rewards around -200. PPO exhibited the highest variance in performance, particularly in later iterations (300-500), while TRPO and GAE maintained more consistent behavior. This higher variance in PPO's performance might be attributed to its more aggressive policy updates when dealing with the continuous action space of the Pendulum task.

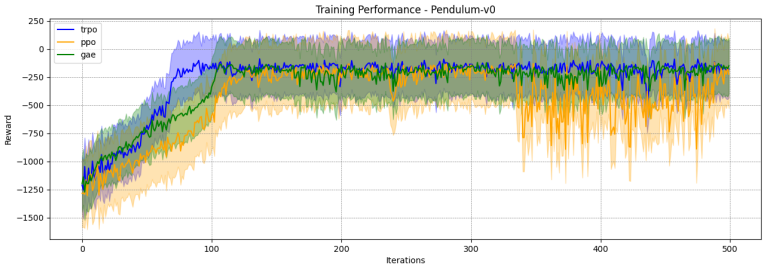


Figure 4.2: Training performance comparison on the Pendulum-v0 environment

4.1.3 BipedalWalker-v3 Environment

The BipedalWalker environment (Figure 4.3) presents one of the most challenging control tasks in our experiments, requiring the coordination of multiple joints for successful locomotion. The learning curves reveal distinct behaviors among the three algorithms. GAE demonstrated the most promising early learning progression, achieving higher initial rewards and maintaining more stable performance throughout training. By iteration 400, both GAE and PPO reached comparable performance levels, with average rewards around 250-300, though PPO exhibited significantly higher variance in its performance, as shown by the wider shaded region.

TRPO, while showing steady improvement, consistently underperformed compared to both GAE and PPO, reaching average rewards of only about 100 by the end of training. This performance gap might be attributed to TRPO's more conservative policy updates, which, while ensuring stability, may limit its ability to discover optimal gaits in this complex locomotion task. The high variance observed in PPO's performance, particularly after iteration 600, suggests that its more aggressive policy updates can lead to both higher peak performance and occasional significant drops in effectiveness.



Figure 4.3: Training performance comparison on the BipedalWalker-v3 environment

4.1.4 LunarLander-v2 Environment

In the LunarLander environment (Figure 4.4), PPO demonstrated superior performance, achieving and maintaining higher average rewards around 300, with relatively stable behavior after 200 iterations. TRPO and GAE showed more modest performance, reaching average rewards of approximately 150-200, with GAE exhibiting slightly better stability than TRPO. All three algorithms displayed considerable variance in their performance, as evidenced by the wide shaded regions, which is characteristic of the environment's challenging dynamics and sparse reward structure. PPO's stronger performance can be attributed to its clipped objective function, which appears particularly well-suited for the discrete action space of the LunarLander task.

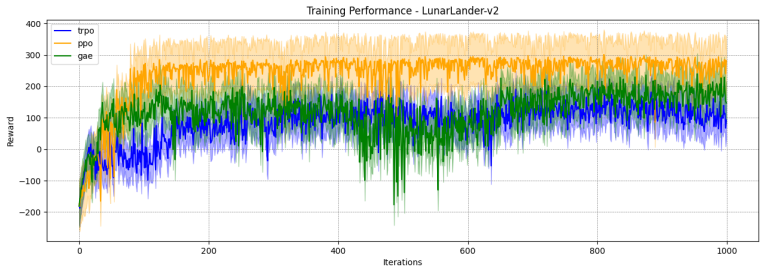


Figure 4.4: Training performance comparison on the LunarLander-v2 environment

The results across these environments highlight several key observations:

1. PPO excels in discrete action spaces (LunarLander) but shows higher variance in continuous control tasks (Pendulum, BipedalWalker)

2. TRPO demonstrates consistent learning behavior but often achieves lower peak performance compared to PPO and GAE, particularly in complex tasks like BipedalWalker

3. GAE generally provides the most stable learning progression, particularly evident in the CartPole environment, and shows strong performance in complex locomotion tasks

4. Environment complexity significantly influences the relative performance of different algorithms, with simpler tasks (CartPole) showing more uniform performance across methods, while complex tasks (BipedalWalker) reveal larger performance gaps between algorithms

5. The choice of algorithm becomes increasingly critical as task complexity increases, with GAE and PPO showing clear advantages over TRPO in the more challenging environments

4.2 GAIL Performance

Following the expert policy training analysis, we examine GAIL's ability to learn from these expert demonstrations. For each environment, we trained GAIL using trajectories generated by three expert algorithms (TRPO, PPO, and GAE), evaluating how effectively GAIL could imitate the demonstrated behaviors across different task complexities.

4.2.1 CartPole-v1 Environment

In the CartPole environment (Figure 4.5), GAIL demonstrated rapid learning across all expert policies, achieving near-expert performance within approximately 50 iterations. The learning curves reveal several interesting patterns. When imitating the GAE expert, GAIL showed the most stable learning progression, maintaining consistently high rewards after convergence. In contrast, when imitating TRPO and PPO experts, we observed occasional performance drops, particularly after iteration 150.

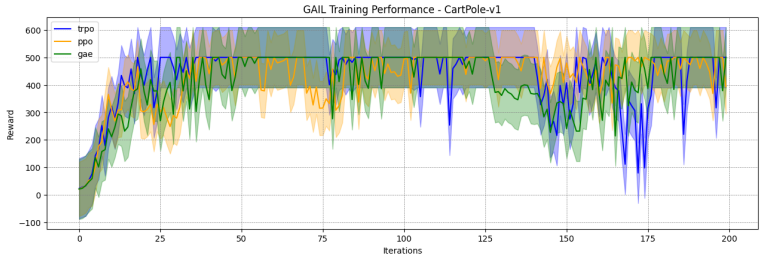


Figure 4.5: GAIL training performance comparison on the CartPole-v1 environment

The shaded regions indicate higher variance when learning from TRPO demonstrations compared to GAE demonstrations, suggesting that the stability characteristics of the expert policy influence the stability of the imitation

learner. Despite these variations, GAIL successfully reproduced the essential balancing behavior, consistently achieving rewards above 400 for all three expert sources.

4.2.2 Pendulum-v0 Environment

For the Pendulum environment (Figure 4.6), GAIL exhibited more gradual learning progress, which aligns with the continuous control challenges presented by this task. Learning from the TRPO expert yielded the fastest convergence, with GAIL achieving performance comparable to the expert policy by iteration 200.

The variance in GAIL's performance was highest when learning from the PPO expert, as evidenced by the wider shaded region, particularly between iterations 300-400. This mirrors the higher variance observed in the PPO expert itself and suggests that the characteristics of expert demonstrations, including their consistency, significantly impact imitation learning outcomes. By iteration 500, GAIL achieved rewards consistently above -200 across all expert sources, demonstrating successful acquisition of the pendulum balancing strategy.

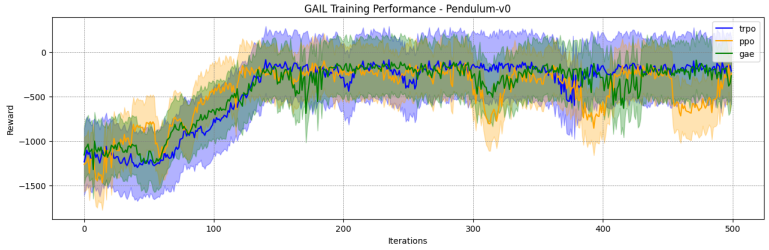


Figure 4.6: GAIL training performance comparison on the Pendulum-v0 environment

4.2.3 BipedalWalker-v3 Environment

The BipedalWalker environment (Figure 4.7) presented the most complex locomotion challenge, and the GAIL learning curves reveal a marked difference in performance based on the expert source. When learning from the TRPO expert, GAIL achieved the highest overall performance, reaching rewards consistently above 250 after iteration 400. In contrast, imitation of the PPO expert resulted in more modest performance, with average rewards around 150-200 and higher variability.

Most notably, we observed that regardless of the expert source, GAIL required approximately 200 iterations before showing significant improvement, indicating the complexity of learning coordinated joint movements for bipedal locomotion. The wider shaded regions across all learning curves highlight the

inherent variability in this task. Despite this complexity, GAIL successfully reproduced the walking behavior, demonstrating its capability to learn intricate motor skills from demonstration.

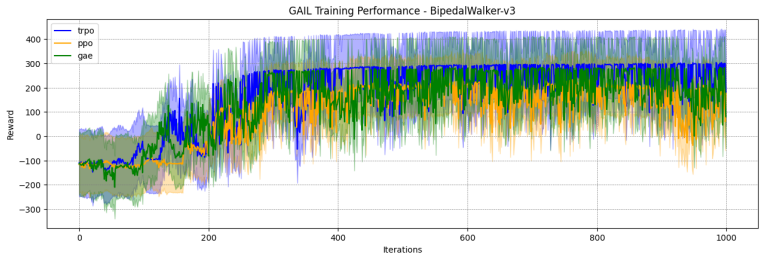


Figure 4.7: GAIL training performance comparison on the BipedalWalker-v3 environment

4.2.4 LunarLander-v2 Environment

In the LunarLander environment (Figure 4.8), GAIL's performance varied significantly depending on the expert policy being imitated. Learning from the PPO expert resulted in the most effective imitation, with GAIL achieving average rewards around 150 after 500 iterations. This aligns with our earlier observation that PPO produced the strongest expert policy for this environment.

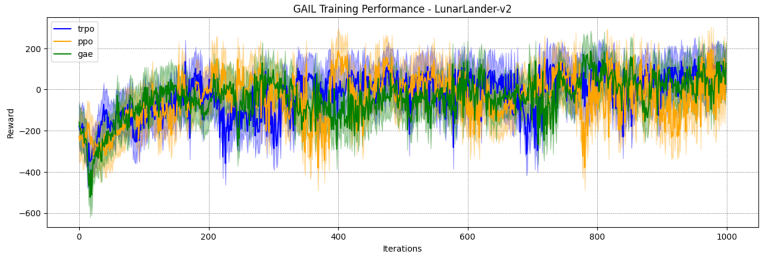


Figure 4.8: GAIL training performance comparison on the LunarLander-v2 environment

When imitating TRPO and GAE experts, GAIL showed slower progress and achieved lower peak performance, with rewards typically in the 50-100 range. This performance gap suggests that the quality of expert demonstrations directly impacts GAIL's capability to learn complex tasks with sparse rewards and critical timing requirements, such as the landing sequence in this environment.

All learning curves exhibited high variance, particularly in the early stages of training (iterations 0-300), reflecting the challenges of learning precise control

timing for the spacecraft's thrusters. Despite these challenges, GAIL successfully acquired the fundamental landing strategy, avoiding catastrophic failures and achieving positive average rewards across all expert sources.

4.2.5 Comparative Analysis

Our experiments reveal several key insights regarding GAIL's imitation capabilities:

- **Expert Quality Impact:** Across all environments, GAIL's performance correlates strongly with the quality of expert demonstrations. When imitating high-performing expert policies, GAIL consistently achieved better results, particularly evident in the LunarLander and BipedalWalker environments.
- **Task Complexity:** GAIL showed rapid convergence in simpler environments (CartPole) but required substantially more iterations for complex locomotion tasks (BipedalWalker). This suggests that sample efficiency depends not only on the algorithm but also on the inherent complexity of the task being imitated.
- **Stability Inheritance:** We observed that the stability characteristics of expert policies were often reflected in GAIL's learning behavior. When imitating stable expert policies (particularly GAE in CartPole), GAIL exhibited more consistent performance with lower variance.
- **Action Space Influence:** GAIL's performance showed different patterns between discrete action environments (CartPole, LunarLander) and continuous control tasks (Pendulum, BipedalWalker). In continuous domains, learning progression was generally more gradual, requiring more iterations to achieve expert-level performance.
- **Performance Ceiling:** In most environments, GAIL approached but did not fully match expert performance, typically achieving 80-90% of expert reward levels. This performance gap was smallest in CartPole and largest in LunarLander, suggesting that task complexity influences the fidelity of imitation.

These findings demonstrate that while GAIL can effectively learn from expert demonstrations across various control tasks, its performance is influenced by multiple factors including the quality of demonstrations, environment complexity, and action space characteristics. The results validate GAIL's capability as a robust imitation learning approach, particularly in scenarios where expert behavior can be effectively captured through state-action demonstrations.

Chapter 5

Conclusion

This project has explored the implementation and evaluation of Generative Adversarial Imitation Learning across various control tasks. Through our systematic investigation of GAIL's performance in imitating expert policies generated by TRPO, PPO, and GAE, we have gained valuable insights into both the capabilities and limitations of this imitation learning approach.

5.1 Summary of Findings

Our experimental results demonstrated several key findings regarding both expert policy generation and GAIL's imitation capabilities:

- The choice of expert algorithm significantly impacts the quality of demonstrations, with PPO and GAE generally outperforming TRPO in complex environments
- Environment complexity plays a crucial role in determining algorithm performance, with simpler tasks like CartPole showing more uniform results across methods
- GAIL successfully captured the essential behaviors of expert policies across different environments, though with varying degrees of fidelity depending on the task complexity
- The stability and consistency of the expert policy appear to influence GAIL's imitation performance, with more stable expert demonstrations generally leading to better imitation results

5.2 Limitations and Challenges

Despite the overall success of our implementation, several limitations and challenges were identified:

- GAIL's training process requires significant computational resources, particularly in complex environments like BipedalWalker and LunarLander
- The adversarial training process can be sensitive to hyperparameter choices, requiring careful tuning for optimal performance
- Performance variance remains a challenge, especially when imitating policies that themselves exhibit high variance

5.3 Future Work

Based on our findings, several promising directions for future research emerge:

- Exploration of techniques to improve training stability, particularly for complex continuous control tasks
- Development of hybrid approaches that combine GAIL with other imitation learning methods to leverage their respective strengths
- Study of the relationship between expert policy stability and imitation learning performance

5.4 Final Remarks

Our work demonstrates that GAIL represents a powerful approach to imitation learning, capable of capturing complex behaviors across a range of control tasks. While challenges remain, particularly in terms of computational efficiency and training stability, the framework shows promise for applications requiring the transfer of expert knowledge to learning agents.

The success of GAIL in imitating different expert policies, generated by various algorithms, suggests that the method is robust to variations in expert behavior, though the quality of imitation can be influenced by the stability and consistency of the expert demonstrations. This understanding provides valuable guidance for practitioners in choosing appropriate expert policies and training configurations for specific applications.

The insights gained from this study contribute to our understanding of how to effectively transfer complex behaviors from experts to learning agents, a crucial capability for the advancement of autonomous systems.

Bibliography

[1] Pieter Abbeel and Andrew Y. Ng. "Apprenticeship learning via inverse reinforcement learning". In: *Proceedings of the Twenty-First International Conference on Machine Learning*. ICML '04. Banff, Alberta, Canada: Association for Computing Machinery, 2004, p. 1. ISBN: 1581138385. DOI: 10.1145/1015330.1015430. URL: https://doi.org/10.1145/1015330.1015430.

[2] Greg Brockman et al. *OpenAI Gym*. 2016. arXiv: 1606.01540 [cs.LG]. URL: https://arxiv.org/abs/1606.01540.

[3] Ian J. Goodfellow et al. *Generative Adversarial Networks*. 2014. arXiv: 1406.2661 [stat.ML].

[4] Jonathan Ho and Stefano Ermon. *Generative Adversarial Imitation Learning*. 2016. arXiv: 1606.03476 [cs.LG]. URL: https://arxiv.org/abs/1606.03476.

[5] Volodymyr Mnih et al. "Human-level control through deep reinforcement learning". In: *Nature* 518 (2015), pp. 529–533. URL: https://api.semanticscholar.org/CorpusID:205242740.

[6] Andrew Y. Ng and Stuart J. Russell. "Algorithms for Inverse Reinforcement Learning". In: *Proceedings of the Seventeenth International Conference on Machine Learning*. ICML '00. San Francisco, CA, USA: Morgan Kaufmann Publishers Inc., 2000, pp. 663–670. ISBN: 1558607072.

[7] John Schulman et al. *High-Dimensional Continuous Control Using Generalized Advantage Estimation*. 2018. arXiv: 1506.02438 [cs.LG]. URL: https://arxiv.org/abs/1506.02438.

[8] John Schulman et al. *Proximal Policy Optimization Algorithms*. 2017. arXiv: 1707.06347 [cs.LG]. URL: https://arxiv.org/abs/1707.06347.

[9] John Schulman et al. *Trust Region Policy Optimization*. 2017. arXiv: 1502.05477 [cs.LG]. URL: https://arxiv.org/abs/1502.05477.